Title

Drawing A House Using OpenGL

Introduction

Computer graphics is a branch of computer science that deals with the creation and display of visual images using computers. Graphical objects are constructed using basic output primitives such as points, lines, and polygons. These primitives act as the building blocks for creating complex structures and scenes used in applications such as computer-aided design, simulations, animation, and graphical user interfaces.

OpenGL (Open Graphics Library) is a widely used graphics programming library that provides functions for designing and displaying two-dimensional and three-dimensional graphics. In this experiment, a colorful two-dimensional house is designed using OpenGL output primitives. Various geometric shapes such as polygons and triangles are combined to represent the walls, roof, windows, and doors of the house. The experiment demonstrates the practical use of output primitives in creating meaningful graphical objects.

Contents

Shapes Used

- Rectangles (Walls, Windows, and Doors)
- Triangle (Roof)
- Quadrilateral Polygon (Upper Roof)

Functions Used

- **glClear(GL_COLOR_BUFFER_BIT)**
  Clears the screen using the predefined background color.
- **glClearColor(r, g, b, a)**
  Sets the background color of the display window. In this program, white color is used: glClearColor(1.0, 1.0, 1.0, 1.0);
- **glColor3f(r, g, b)**
  Sets the current drawing color using RGB values between 0 and 1.
- **glBegin() and glEnd()**
  Mark the beginning and end of drawing primitives.
- **GL_POLYGON**
  Used to draw polygons such as walls, windows, doors, and roof sections.
- **GL_TRIANGLES**
  Used to draw the triangular roof of the house.
- **glVertex3f(x, y, z)**
  Specifies the coordinates of a vertex in three-dimensional space.
- **glMatrixMode(mode)**
  Selects the current matrix mode. In this program, the projection matrix is used.
- **glLoadIdentity()**
  Replaces the current matrix with the identity matrix.

- **glOrtho(left, right, bottom, top, near, far)**
  Defines a two-dimensional orthographic viewing region. In this program:
  glOrtho(0.0, 1.0, 0.0, 1.0, -1.0, 1.0);
- **glutSwapBuffers()**
  Swaps the front and back buffers for smooth display using double buffering.

GLUT Functions

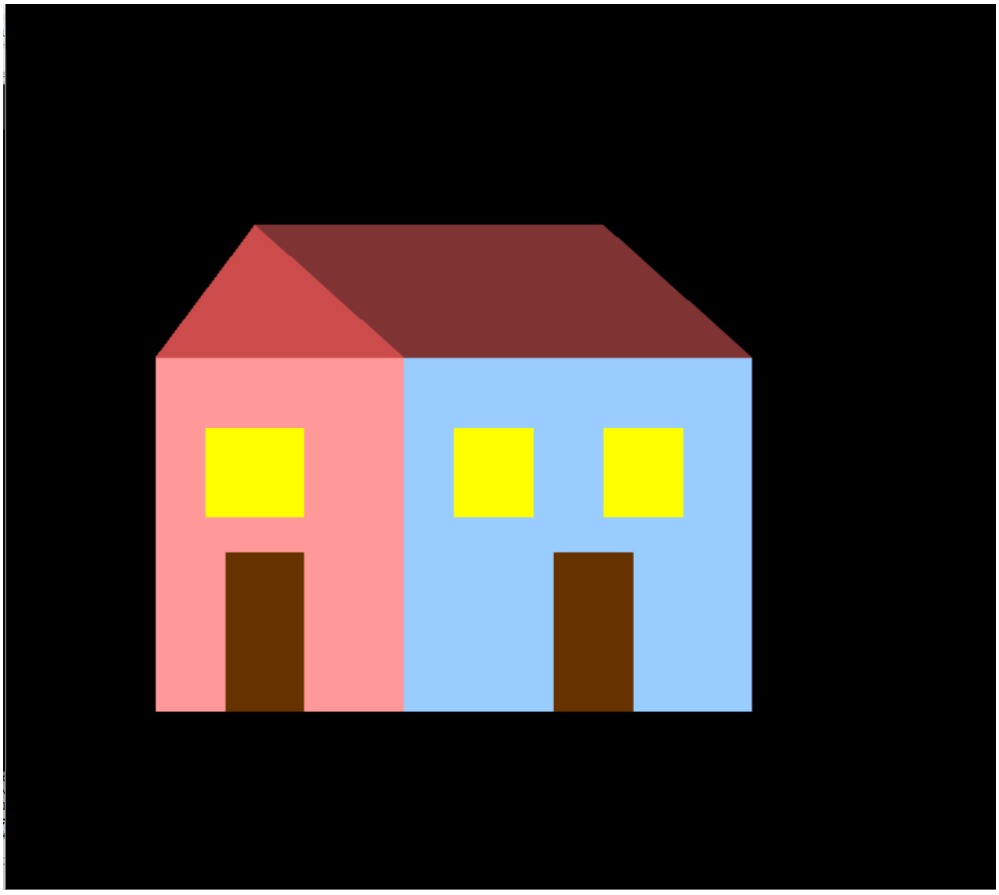
- **glutInit()**
  Initializes the GLUT library.
- **glutInitDisplayMode()**
  Sets the display mode. In this program, RGB color mode and double buffering are used.
- **glutInitWindowPosition()**
  Specifies the initial position of the display window.
- **glutInitWindowSize()**
  Sets the size of the display window.
- **glutCreateWindow()**
  Creates a display window with a specified title.
- **glutDisplayFunc()**
  Registers the display callback function responsible for drawing the house.
- **glutMainLoop()**
  Enters the event-processing loop and keeps the window active.

Code:

```
#include <GL/glut.h>
#include <GL/gl.h>

void init()
{
    glClearColor(0.0, 0.0, 0.0, 0.0);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    glOrtho(0.0, 1.0, 0.0, 1.0, -1.0, 1.0);
}

void Draw()
{
    glClear(GL_COLOR_BUFFER_BIT);

    // LEFT HOUSE
    glColor3f(1.0, 0.6, 0.6);
    glBegin(GL_POLYGON);
        glVertex3f(0.15, 0.20, 0.0);
        glVertex3f(0.40, 0.20, 0.0);
        glVertex3f(0.40, 0.60, 0.0);
        glVertex3f(0.15, 0.60, 0.0);
    glEnd();

    // RIGHT HOUSE
    glColor3f(0.6, 0.8, 1.0);
    glBegin(GL_POLYGON);
        glVertex3f(0.40, 0.20, 0.0);
        glVertex3f(0.75, 0.20, 0.0);
        glVertex3f(0.75, 0.60, 0.0);
        glVertex3f(0.40, 0.60, 0.0);
    glEnd();

    // LEFT TRIANGLE ROOF
    glColor3f(0.8, 0.3, 0.3);
    glBegin(GL_TRIANGLES);
        glVertex3f(0.15, 0.60, 0.0);
        glVertex3f(0.25, 0.75, 0.0);
        glVertex3f(0.40, 0.60, 0.0);
    glEnd();

    // TOP ROOF
    glColor3f(0.5, 0.2, 0.2);
    glBegin(GL_POLYGON);
        glVertex3f(0.25, 0.75, 0.0);
        glVertex3f(0.60, 0.75, 0.0);
        glVertex3f(0.75, 0.60, 0.0);
        glVertex3f(0.40, 0.60, 0.0);
    glEnd();

    // LEFT WINDOW
    glColor3f(1.0, 1.0, 0.0);
    glBegin(GL_POLYGON);
        glVertex3f(0.20, 0.42, 0.0);
```

```
        glVertex3f(0.30, 0.42, 0.0);
        glVertex3f(0.30, 0.52, 0.0);
        glVertex3f(0.20, 0.52, 0.0);
    glEnd();

    // RIGHT WINDOW 1
    glBegin(GL_POLYGON);
        glVertex3f(0.45, 0.42, 0.0);
        glVertex3f(0.53, 0.42, 0.0);
        glVertex3f(0.53, 0.52, 0.0);
        glVertex3f(0.45, 0.52, 0.0);
    glEnd();

    // RIGHT WINDOW 2
    glBegin(GL_POLYGON);
        glVertex3f(0.60, 0.42, 0.0);
        glVertex3f(0.68, 0.42, 0.0);
        glVertex3f(0.68, 0.52, 0.0);
        glVertex3f(0.60, 0.52, 0.0);
    glEnd();

    // LEFT DOOR
    glColor3f(0.4, 0.2, 0.0);
    glBegin(GL_POLYGON);
        glVertex3f(0.22, 0.20, 0.0);
        glVertex3f(0.30, 0.20, 0.0);
        glVertex3f(0.30, 0.38, 0.0);
        glVertex3f(0.22, 0.38, 0.0);
    glEnd();

    // RIGHT DOOR
    glBegin(GL_POLYGON);
        glVertex3f(0.55, 0.20, 0.0);
        glVertex3f(0.63, 0.20, 0.0);
        glVertex3f(0.63, 0.38, 0.0);
        glVertex3f(0.55, 0.38, 0.0);
    glEnd();

    glutSwapBuffers();
}

int main(int argc, char **argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB | GLUT_DOUBLE);
    glutInitWindowPosition(100, 100);
    glutInitWindowSize(600, 600);

    glutCreateWindow("House");

    init();
    glutDisplayFunc(Draw);

    glutMainLoop();

    return 0;
}
```

Output:



Discussion

In this experiment, a colorful 2D house was successfully designed using OpenGL output primitives. The house consists of two connected building sections with a roof, three windows, and two doors. Different colors were applied to the walls, roof, windows, and doors to improve the visual appearance of the structure. The design was created using geometric shapes such as polygons and triangles, demonstrating how simple output primitives can be combined to form meaningful graphical objects. The successful execution of the program verifies the effectiveness of OpenGL in creating 2D graphics and enhances the understanding of coordinate-based drawing techniques and graphical object construction.